

# Lab 2: Pipelining and Superscalar (Detailed Instructor's Guide)

Instructor Copy with Verified Results

October 16, 2025

## Part 1: Increasing Functional Units (In-Order)

### Configuration C

- **Changes:** Compared to the baseline in-order processor from Lab 1 (`config_a.cfg`), the number of functional units (`-res:*`) was increased from 1 to 4. The pipeline remains in-order and single-issue.
  - **Impact of Changes:** In an in-order pipeline, instructions must be issued in their original program sequence. The pipeline's throughput is strictly limited by the narrowest stage in the front-end (fetch, decode, issue). Since the issue width is 1, the processor can only send one instruction to the execution stage per cycle, regardless of how many functional units are available.
  - **Resulting IPC: 0.5071**
  - **Conclusion:** The IPC is identical to the baseline in-order processor. The extra functional units remain idle because the front-end cannot supply them with instructions. This demonstrates that simply adding execution resources without widening the pipeline front-end yields no performance gain in a simple in-order machine. The bottleneck is the issue width.
- 

## Part 2: Increasing Functional Units (Out-of-Order)

### Configuration D

- **Changes:** This configuration takes the resource-rich design from Part 1 and enables out-of-order execution (`-issue:inorder false`). However, the pipeline width remains narrow (`decode=1, issue=1`).
- **Impact of Changes:** Enabling out-of-order execution allows the processor to look for independent instructions within the instruction window (RUU) and execute them even if an earlier instruction is stalled. This can hide some latency from data dependencies.
- **Resulting IPC: 0.5184**
- **Conclusion:** There is a very small IPC improvement over the in-order version (0.5071). While OOO execution helps, the performance is still severely choked by the single-issue front-end, which starves the execution units.

### Bottleneck Analysis

- **Changes:** The student is asked to widen the decode and issue widths to 4, one at a time, to find the bottleneck.
- **Impact of Changes:**
  - Setting `-decode:width 4` while `-issue:width 1` has no effect. The issue stage can still only accept one instruction per cycle from the decoder.
  - Setting `-issue:width 4` while `-decode:width 1` also has no effect. The decoder can still only supply one instruction per cycle to the issue stage.

- **Resulting IPC (for both experiments): 0.5184**
  - **Conclusion:** The bottleneck is the entire pipeline front-end. Performance does not increase because the narrowest stage dictates the overall throughput. To see a benefit, both decode and issue widths must be increased together.
- 

## Part 3: Widening the Pipeline (Few Resources)

### Configuration E

- **Changes:** This configuration models a wide out-of-order pipeline (fetch=4, decode=4, issue=4) but with only one of each functional unit (-res:\* 1).
  - **Impact of Changes:** The wide front-end can fetch, decode, and issue up to 4 instructions per cycle. However, with only one ALU, one multiplier, etc., most of these instructions will immediately stall, waiting for a free execution unit. This creates a massive bottleneck due to **structural hazards**.
  - **Resulting IPC: 0.5184**
  - **Conclusion:** The IPC is identical to the narrow out-of-order pipeline from Lab 1 (config.b.cfg). This proves that widening the front-end is useless if there are no execution resources to handle the increased instruction flow. It highlights the need for a balanced processor design.
- 

## Part 4: Wide Pipeline and Many Resources

### Configuration F

- **Changes:** This configuration combines the wide front-end from Part 3 with the plentiful resources from Part 2. This represents a balanced, 4-way superscalar out-of-order processor.
  - **Impact of Changes:** By removing both the front-end bottleneck and the structural hazard bottleneck, the processor is now able to exploit significant Instruction-Level Parallelism (ILP). It can fetch, issue, and execute multiple instructions in parallel.
  - **Resulting IPC: 1.3035**
  - **Conclusion:** A dramatic increase in IPC is observed. This demonstrates the synergistic effect of a wide pipeline and sufficient parallel execution units, which is the foundation of modern high-performance CPUs.
- 

## Part 5: Increasing the Instruction Window

### Configuration G

- **Changes:** This configuration takes the balanced superscalar design from Part 4 and doubles the size of the instruction window by increasing the Register Update Unit size (-ruu:size) from 8 to 16.
- **Impact of Changes:** The RUU holds instructions that have been decoded but are waiting to be issued or have completed execution but are waiting to be committed. A larger RUU allows the processor to "see" further into the program's instruction stream. This increases its ability to find independent instructions to execute out of order, which is especially effective at hiding the latency of long operations like memory loads that miss the cache.

- **Resulting IPC: 1.3245**
- **Conclusion:** The IPC increases further, though not as dramatically as in the previous step. This shows that once a processor is reasonably balanced, increasing the instruction window size allows it to better exploit the available ILP, leading to another level of performance improvement.